Distributed Job Scheduler

System Design & Implementation Report

Submission for Distributed Job Scheduling Platform Evaluation

sudhansudharsan3680@gmail.com

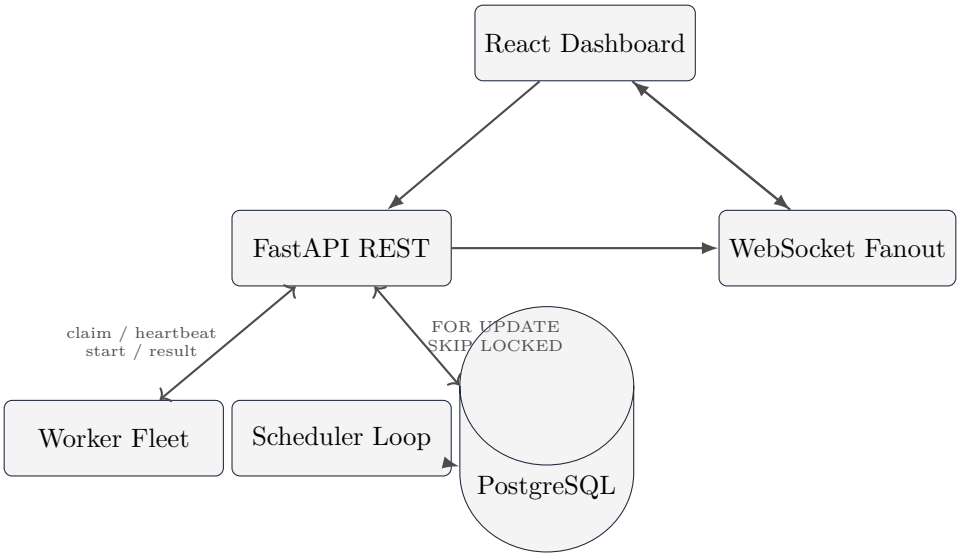
2026

Repository: https://github.com/sudharsan3680/distributed-job-scheduler **Stack:** FastAPI · SQLAlchemy 2.0 · PostgreSQL · Alembic · React/TS · WebSocket

1. Executive Summary

A production-inspired, multi-tenant distributed job scheduling platform that reliably executes asynchronous background jobs across a fleet of competing workers. PostgreSQL acts as both the system of record and the work queue; the API is stateless and horizontally scalable; workers are independent processes that claim work over HTTP. The platform covers the full requirement set: authentication and project/queue management, all job types (immediate, delayed, scheduled, cron, batch), atomic claiming, the complete job lifecycle with retries and a dead-letter queue, configurable backoff, execution logs, and a live monitoring dashboard.

2. System Architecture



The database is the single source of truth and the only component that must be made highly available. Workers never touch the database directly, so there is no dual-source-of-truth consistency problem. Postgres-as-coordinator (instead of a Redis/RabbitMQ broker) trades a lower throughput ceiling for

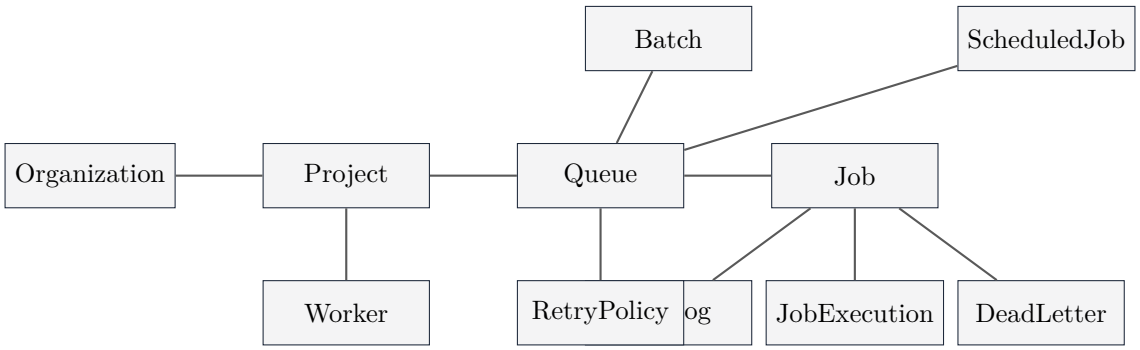
one piece of infrastructure to run and reason about; queue sharding is the documented next step if that ceiling is reached.

3. Requirements Coverage

Requirement	Status / Where
Auth + project/queue management	JWT users + orgs; projects scoped by API key; RBAC (owner/admin/member/viewer) enforced on all mutating routes.
Queue config	priority, max_concurrency, retry policy, pause/resume, per-queue rate limit, statistics.
All job types	immediate, delayed, scheduled, recurring (cron via `croniter`), batch (up to 1000).
Worker service	polls, atomic claim, concurrent execution (`Semaphore`), heartbeats, graceful drain.
Lifecycle	QUEUED → SCHEDULED → CLAIMED → RUNNING → COMPLETED, with FAILED + DEAD_LETTER.
Retries	fixed / linear / exponential backoff with cap + full jitter.
Observability	job_executions (per attempt), job_logs, worker heartbeats, dashboard metrics.
Dashboard	queue health, worker status, job explorer, execution logs, retry/replay, throughput chart.

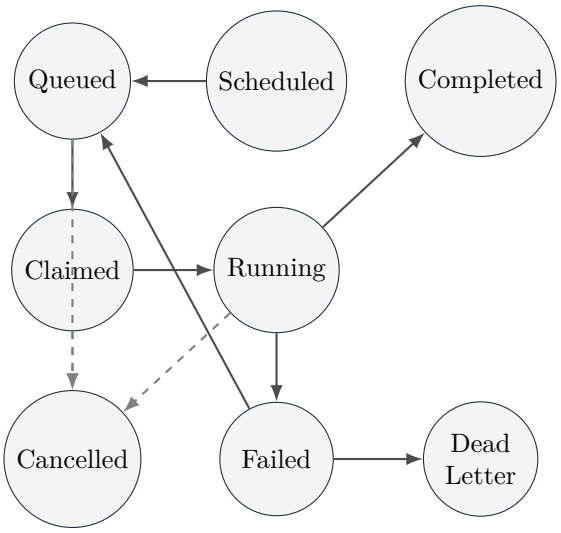
4. Database Design

3NF relational schema, surrogate BIGINT PKs; natural keys carry their own UNIQUE constraints. Scheduling definitions (`scheduled_jobs`) are separate from job instances (`jobs`); `dead_letter_queue` is its own table (narrow DLQ scan + frozen payload snapshot).



Key indexes: `ix_jobs_claim_scan(queue_id, status, run_at)` (the atomic claim scan), `ix_scheduled_jobs_due(is_active, next_run_at)`, `ix_heartbeats_worker_time`, `ix_job_logs_job_created`. Cascades: org → project → queue → job → executions/logs (CASCADE); user/worker → attribution (SET NULL) so audit history survives deletions.

5. Job Lifecycle



A retriable failure lands in FAILED (visible in the dashboard) until its backoff elapses, then the claim path promotes it back to QUEUED. Once attempts are exhausted the job moves to DEAD_LETTER with a frozen payload snapshot.

6. Reliability & Concurrency

- **Atomic claiming:** `SELECT …FOR UPDATE SKIP LOCKED` plus an in-process per-queue `asyncio.Lock` (closes same-process interleaving); committed inside the lock.
- **Lease renewal:** worker heartbeats extend `lease_expires_at` on held jobs, so a long-running job is never prematurely reaped / double-executed.
- **Stale-lease reaper:** requeues (or dead-letters) jobs whose lease expired; closes the dangling execution row as `TIMED_OUT`.
- **FAILED is real:** retriable failures sit in FAILED until backoff elapses, then the claim path promotes them to QUEUED (dashboard "Failed" count is meaningful).
- **Idempotency:** `UNIQUE(queue_id, idempotency_key)`; manual DLQ replay resets the attempt budget so replay actually retries.
- **Graceful shutdown:** SIGTERM/SIGINT → stop claiming, mark DRAINING, finish in-flight work (up to a timeout), deregister.

7. API Surface (selected)

Group	Endpoints
Auth	`POST /auth/register`, `POST /auth/login`
Projects	`POST/GET /organizations/{org}/projects` (returns API key once)
Queues	CRUD, `patch`, `pause/resume`, `stats`
Jobs	create (1 or batch), list (paginated/filtered), detail, `cancel`, `retry`, scheduled-jobs, dead-letter-queue
Workers	`register`, `heartbeat`, `claim`, `start`, `result`, `drain`, `deregister`
Dashboard	`health`, `queues`, `workers` (JWT); `WS /ws/projects/{id}?token=JWT`

All errors are structured: `{error:{code,message,details}}`; list endpoints are paginated.

8. Testing & Quality

- Lifecycle, retry-then-DLQ, queue pause, dependency ordering, RBAC (VIEWER blocked / OWNER allowed).
- Real `FAILED` state + backoff promotion; scheduler promote/reap (requeue + TIMED_OUT, and dead-letter when retries exhausted).
- Postgres-gated concurrency race: two workers, no duplicate/lost claims under real `SKIP LOCKED`.
- Structured error handling, JWT auth fail-fast in production, CORS hardened, WebSocket auth.

9. Running It

```
docker compose up --build        # API :8000, Frontend :5173, Postgres :5432
# register -> create project (save the shown API key) -> start workers:
python -m app.worker.worker --base-url http://localhost:8000 \
  --api-key <PROJECT_API_KEY> --project-id 1 --queues default --concurrency 4
```

10. Trade-offs / Future Work

In-process rate-limit & WebSocket fanout (fine for one replica; Redis/LISTEN-NOTIFY at multi-replica); scheduler not leader-elected (idempotent, redundant today); no soft-delete; no event-driven push (`LISTEN/NOTIFY`); queue sharding and AI failure summaries identified as next steps.